CS 211: Object Oriented Programming - Spring 2026

Programming Assignment 6 – Connect-N

Learning Objectives

In this assignment, you will:

- Work with 15 classes (7 already complete, 8 new or partially complete). Those do not include the tester files provided.
- Design inheritance relationships and write fields, methods, and constructors for your classes
- Complete several tasks to gain hands-on experience with OOP
- Practice using OOP, Exceptions, Generics, and Collections to solve the problems presented
- Implement a recursive algorithm (minimax) to create an AI opponent

Overview

In this assignment, you will apply concepts learned throughout the semester, including enumeration, inheritance, interfaces, collections, recursion, and input/output, to build a **Connect-N** game.

What is Connect-N?

Connect-N is our fun CS 211 generalization of the classic Connect-Four game. In traditional Connect Four, two players take turns dropping colored discs into a vertical grid, and the first player to connect four discs in a row (horizontally, vertically, or diagonally) wins. In our Connect-N variant, the number of discs needed to win (N) is configurable, as are the board dimensions.

Take some time to read about Connect Four and consider playing a few games to better understand the mechanics before you begin coding.

Learn more about Connect Four (or 4-in-a-line, or Straight-4) https://en.wikipedia.org/wiki/Connect_Four

Optionally, play some games online: https://www.mathsisfun.com/games/connect4.html

Project Structure

Files Provided (Do NOT Modify)

The following files are already complete. Do not modify them.

File	Description
`Avatar.java`	An Enum containing all possible avatars a player can choose from. We use a graphical user interface (GUI) to run this game, and the avatar image displayed depends on the constant used when initializing a Player object.  The corresponding PNG files are located in the **resources** folder.
`Displayable.java`	A functional interface (an interface with a single method declaration). Any class representing an object with a `getDisplayName()` method must implement this interface. In this project, the `Player` class implements `Displayable`.
`DifficultyLevel.java`	An Enum representing the AI difficulty levels (e.g., EASY, MEDIUM, HARD, IMPOSSIBLE). The difficulty determines how many moves ahead the computer simulates.

`ConnectNUI.java`
`GamePanel.java`
`SidePanel.java`
`PlayerSetup.java`
	The graphical user interface related classes for playing the game. Run `ConnectNUI` to play once all classes are complete.

Files You Will Create (3 files)

Create these Java classes from scratch:

File	Description
`MyArrayList.java`	A custom implementation of an ArrayList data structure. You can only import: - `java.util.Iterator`.
`MyIterator.java`	A custom Iterator implementation that can be used to make MyArrayList iterable. You can only import: - `java.util.Iterator;` - `java.util.NoSuchElementException;`
`Utility.java`	A utility class containing some helper methods for the game. You can only import: - `java.io.File;` - `java.io.FileNotFoundException;` - `java.util.Scanner;`

Note: Wait until you read the implementation guide before you start writing code.

Files You Will Complete (5 files)

Templates are provided for these classes. Complete the required methods. The template will indicate what method to complete. Those are indicated by the "TO-DO" flag in the comment before the method you need to complete or implement from scratch.

File	Description
`Player.java`	Abstract base class representing a game player. ***No imports allowed.***
`HumanPlayer.java`	Subclass of Player representing a human player. ***No imports allowed.***

`ComputerPlayer.java`	Subclass of Player representing the AI opponent. ***No imports allowed.***
`Board.java`	Represents the game board and provides board-related utilities. ***No imports allowed.***
`Engine.java`	The game engine that manages turns, validates moves, and implements the AI logic. ***No imports allowed.***

Instructions

• **Download** `PA6.zip` file from Piazza. It contains a folder called `yourCodeHere`. This will be your project directory.

• **Create** the 3 Java classes listed above.

• **Complete** the 5 partially implemented Java classes, read carefully all in-method comments as they are also part of the project specification.

• **Test** your code thoroughly:
- Use the provided tester classes
- Add your own cases for more comprehensive coverage

• **Submit** your final versions of all 8 java files through Gradescope.

IMPORTANT NOTES:
- **Do NOT** add any additional methods or attributes (not in the project description/specification) unless they are private.
- **Do NOT** use any import unless specified in the project description.
- **You must Use Recursion when specified. See grading rubric.**

Implementation Guide

We highly recommend implementing the classes in the following order. To make the process manageable, we've organized the work into a proposed daily schedule. Below are high level description of the steps. Keep in mind that comments in the template files provided are also important part of the project specification.

Day 1: Understand the Project

Goals:

- **Read** this entire document carefully
- Use **Piazza** or **office hours** to clarify any questions
- Do not start coding until you **understand the big picture**

Tasks:

- **Read** about the Connect Four game on Wikipedia
- **Play** a few games online to understand the strategy involved
- **Review all provided template files (yes, they contains some more detailed specification)** to understand the project structure and description.

Day 2: Implement MyArrayList and MyIterator

Goals:

- Implement a custom generic ArrayList which is Iterable
- Understand how Iterable and Iterator work together
- Verify correctness using provided testers

Classes to implement (in this order):

- **`MyArrayList.java`**

  Implement a custom generic **`ArrayList`** that implements **`Iterable<T>`**. This class should use an underlying array to store elements. It does not have to support dynamic resizing. For this project, use a fixed capacity of 500. The array will not grow or shrink dynamically.

  IMPORTANT NOTE: In this class, you can only import **`java.util.Iterator`**

  Required Fields:

Field	Description
`private T[] data`	The underlying array to store elements.
`private int size`	The number of elements currently in the list

Required Methods:

Method	Description
`public MyArrayList()`	Default constructor; initializes with a default capacity (use 500 for this project)
`public void add(int index, T element)`	Inserts an element at the specified index; shifts subsequent elements
`public void add(T element)`	Adds an element to the end of the list
`public T get(int index)`	Returns the element at the specified index
`public T remove(int index)`	Removes and returns the element at the specified index
`public int size()`	Returns the number of elements in the list
`public boolean isEmpty()`	Returns true if the list contains no elements
`public boolean contains(T element)`	Returns true if the list contains the specified element. False otherwise.
`public Iterator<T> iterator()`	Returns a **`MyIterator`** instance for this list (required by **`Iterable<T>`**)

Implementation Notes:

- The **`contains()`** methods must use recursion. Hint: You might need a (private) helper method.
- When the array is full and a new element needs to be added, throw **`IllegalStateException`**.
- Throw **`IndexOutOfBoundsException`** for invalid indices.
- The **`iterator()`** method should return a new instance of **`MyIterator`**. So, you may need to implement the **`MyIterator`** class first, then return to finish implementing **`MyArrayList`**.

- **`MyIterator.java`**

  Implement a custom **`Iterator`** that allows traversal of **`MyArrayList`**. This class should implement **`Iterator<T>.`**

  IMPORTANT NOTE: In this class, you can only import:
  `java.util.Iterator`
  `java.util.NoSuchElementException`

  Required Fields:

Field	Description
`private MyArrayList<T> list`	Reference to the list being iterated
`private int currentIndex`	The current position in the iteration

  Required Methods:

Method	Description
`public MyIterator(MyArrayList<T> list)`	Constructor that takes the list to iterate over
`public boolean hasNext()`	Returns true if there are more elements to iterate
`public T next()`	Returns the next element and advances the iterator

  Implementation Notes:

  - The **`next()`** method should throw **`NoSuchElementException`** if there are no more elements.
  - Remember to import **`java.util.Iterator`** and **`java.util.NoSuchElementException`**.
  - Once this class is complete, you should be able to complete the method **`iterator()`** of **`MyArrayList`** class.

Why This Matters: By implementing **`Iterable<T>`**, your **`MyArrayList`** can be used in enhanced for-loops like in this example below:

```
MyArrayList<String> names = new MyArrayList<>();
names.add("CS112");
names.add("CS211");
names.add("CS310");

// This works because MyArrayList implements Iterable<T>
for (String name : names) {
    System.out.println(name);
}
```

Testing:

Run the following testers and ensure all tests pass:

- **`MyArrayListTester.java`**
- **`MyIteratorTester.java`**

Important: If you cannot pass all tests, seek guidance at office hours immediately. These foundational classes are used in the rest of the project.

Day 3: Implement Utility and Player classes

Goals:

- Complete helper methods in the Utility class
- Implement the Player class hierarchy
- Begin implementing the Board class

Classes to implement:

- **Utility.java**

  A utility class containing static helper methods used throughout the project.

  IMPORTANT NOTE: In this class, you can only import:

  `java.io.File`
  `java.io.FileNotFoundException`
  `java.util.Scanner`

Method	Description
`public static MyArrayList<String> loadBadWords(String filename)`	Loads a list of prohibited words from a file and returns them in a **`MyArrayList`**

`public static boolean containsBadWord(String input, String filename)`	Returns true if the input string contains any word from the file (use filename provided as argument)
`public static boolean isEven(int number)`	Returns true if the number is even. ***This must be a recursive method.***
`public static boolean isOdd(int number)`	Returns true if the number is odd. ***This must be a recursive method.***
`public static int countInColumn(int[][] board, int col, int piece)`	Counts how many of the specified piece type are in the given column. ***This must be or use a recursive method.***

Implementation details:

- For **`loadBadWords()`**, use file I/O to read words from a text file (one word per line).
- For **`containsBadWord()`**, consider using your **`MyArrayList`** iterator to check each bad word.
- The methods **`isEven()`** and **`isOdd()`** must be recursive methods (either direct or indirect recursion).
- For **`countInColumn()`**, assume that board is a rectangular 2D array representing a game board. The piece of interest can be any given number (e.g., 0, 1 or 2). Later in the rest of the project, we will use 1 or 2 to represent piece colors. 0 will then represent an empty spot.
- The method **`countInColumn()`** must use recursion also.

- **Player.java**

Note: For this and the remaining Java files, do not add any additional import statements other than the ones provided in the template file.

Method to complete / implement	Description
`public Player(String displayName, Avatar avatar)`	Constructor that initializes a player with a name and avatar. The player's Display Name must be a valid one (length between 2 and 15 inclusive) and no bad word used as part of the Display Name.

Implementation Note:

If the display name is invalid (length not between 2 and 15 inclusive, or contains a bad word), throw an **`IllegalArgumentException`** with an appropriate message.

If **`avatar`** argument is null, use **`Avatar.AVATAR_A`**.

- **HumanPlayer.java**

Method to complete / implement	Description
`public HumanPlayer(String displayName, Avatar avatar)`	Constructor that calls the parent class constructor to initialize the human player with a name and avatar

- **ComputerPlayer.java**

Method to complete / implement	Description
`public ComputerPlayer(String displayName, Avatar avatar)`	Constructor that calls the parent class constructor to initialize the computer player with a name and avatar.

- **Board.java**

The Board class represents the board of the game. One of its attributes is the **`boardState`** which is represented as a 2D int array (**`int[][]`**) where:

`0` = empty cell
`1` = Player 1's piece
`2` = Player 2's piece

Row 0 is the **top of the board**; pieces fall downward.

Method to complete / implement	Description
`public Board(int rows, int cols, int connectN)`	Constructor with custom dimensions and connectN values.
`public Board(int rows, int cols)`	Constructor with custom dimensions
`public Board()`	Default constructor

`public void setRows(int rows)`	Sets the number of rows
`public void setCols(int cols)`	Sets the number of columns
`public void setConnectN(int connectN)`	Sets how many in a row are needed to win
`public static boolean isValidMove(int[][] boardState, int col)`	Checks if a move is valid (column exists and is not full)
`public static int[][] copyAGivenBoardState(int[][] original)`	Creates a deep copy of the board state
`public static int dropPieceSimulated(int[][] b, int col, int piece)`	Simulates dropping a piece; returns the row where it lands
`public static boolean isBoardFullForBoard(int[][] b)`	Checks if the board is completely full

Testing:

Run the following testers:

- `UtilityTester.java`
- `HumanPlayerTester.java`
- `ComputerPlayerTester.java`
- `BoardTester.java`

Additional Tasks:

- Re-read the Board.java and Engine.java templates to understand how all pieces fit together.
- Seek clarification on Piazza or office hours if anything is unclear.

Day 4: Understand the Minimax Algorithm

Goals:

- Learn how the minimax algorithm works
- Understand how to apply it to Connect-N

Background:

The Computer player needs to determine the best move when it is their turn. We use the **minimax algorithm**, a recursive strategy that assumes both players play optimally:

- The **Computer** wants to **maximize** its score
- The **Opponent** wants to **minimize** the Computer's score

This creates a game tree where we evaluate future board states to find the optimal move.

Pseudocode for `findBestMove()`:

```
def findBestMove(board):
    depth = number of moves to look ahead (based on difficulty level)
    computerTurn = true

    [bestMove, eval] = minimax(board, depth, computerTurn)
    return bestMove
```

Pseudocode for `minimax()`:

```
def minimax(board, depth, isMyTurn):

    // Base case: stop recursion when depth is 0 or game is over
    if depth == 0 OR gameIsOver(board):
        return [null, evaluateBoard(board)]


    if isMyTurn:  // Computer's turn - MAXIMIZE score
        maxScore = -INFINITY
        bestMove = null

        for each valid column:
            boardCopy = deepCopy(board)
            dropPiece(boardCopy, column, computerPiece)

            [_, score] = minimax(boardCopy, depth - 1, false)

            if score > maxScore:
                maxScore = score
                bestMove = column

        return [bestMove, maxScore]

    else:  // Opponent's turn - MINIMIZE score
        minScore = +INFINITY
        bestMove = null

        for each valid column:
            Evaluate the score for that move.
            Set minScore and bestMove to correspond to the smallest score
            found so far.


        return [bestMove, minScore]
```

Key Insight:

The opponent's objective is opposite to the Computer's. A higher score favors the Computer; a lower score favors the human player. The minimax algorithm accounts for this by alternating between maximizing and minimizing at each level of recursion.

Tasks:

- **Read** about minimax algorithm (on Wikipedia)
- **Ask questions on Piazza** if anything is unclear
- **Trace through the algorithm** by hand with a small example

Day 5: Implement the Game Engine

Goals:

- **Complete** the Engine class
- **Test the game** by playing against the AI

Class to complete:

- `Engine.java`

Method to complete / implement	Description

`public Engine(`
`  Player player1,`
`  Player player2,`
`  int rows, int cols,`
`  int connectN,`
`  DifficultyLevel difficultyLevel)`
	Constructor that initializes the game engine

`public Engine(`
`  Player player1,`
`  Player player2,`
`  int rows, int cols,`
`  int connectN)`
	Constructor that initializes the game engine

`public int dropPiece(int col, int piece)`	Drops a piece in the specified column; returns the row where it lands
`private boolean checkWinForBoardState(int[][] b, Player player)`	Checks if the specified player has won

`public void findBestMove()`	This method returns the best move (col) and a quantitative evaluation of this move. findBestMove() uses minimax algorithm to determine the best column, then drop the AI's piece in that column.

Testing:

- You have seen enough JUnit Tester to implement your own now. It is optional but recommended to test this file.
- Run **`ConnectNUI`** to play the game
- Experiment with different board sizes, connectN values, and difficulty levels
- At the highest difficulty, you should not be able to beat the AI consistently. If you can, something is wrong!

Day 6: Polish and Test

Goals:

- **Complete** any remaining work on the Engine class
- Perform thorough **testing**
- (Optional) Implement **bonus features described above.**

Tasks:

- **Finish** any incomplete methods
- **Test** edge cases (full board, immediate wins, etc.)
- **Play multiple games** at various difficulty levels

Optional Bonus (5 points)

Improve the efficiency of your minimax algorithm by implementing alpha-beta pruning. This optimization eliminates branches that cannot possibly affect the final decision, significantly speeding up the AI (since it generates less recursive calls).

Resources:

Alpha-Beta Pruning on Wikipedia (https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning)

Running the Game

When all classes are complete, run ConnectNUI.java to play the game. You will be prompted to enter:

- Your display name
- AI difficulty level
- Board dimensions (rows and columns)
- How many in a row to win (N)

Enjoy playing your creation!

Grading rubric

Component	Description
Autograder - **MyLinkedList (20 points)** - **MyIterator (10 points)** - **Utility (30 points)** - **Player (5 points)** - **HumanPlayer (5 points)** - **ComputerPlayer (5 points)** - **Board (10 points)** - **Engine (15 points + 5 Bonus Points)**	100 points
Bonus (MiniMax Alpha-beta pruning)	+ 5 points (via Manual inspection)
Violations such as: **- failing to use recursion when asked to in:** **-> Utility.java isEven()** **-> Utility.java isOdd()** **-> Utility.java countInColumn()** **-> MyArrayList.java contains()** **-> Engine.java findBestMove()** **- using JCF classes to avoid implementing MyArrayList properly, more generally using not allowed imports.**	- 10 points per violation (via Manual inspection)

Getting Help

- Piazza: Post questions (check for existing answers first)
- Office Hours: Get one-on-one help from instructors or TAs
- Start early and ask questions often. Good luck, and have fun building your Connect-N game!

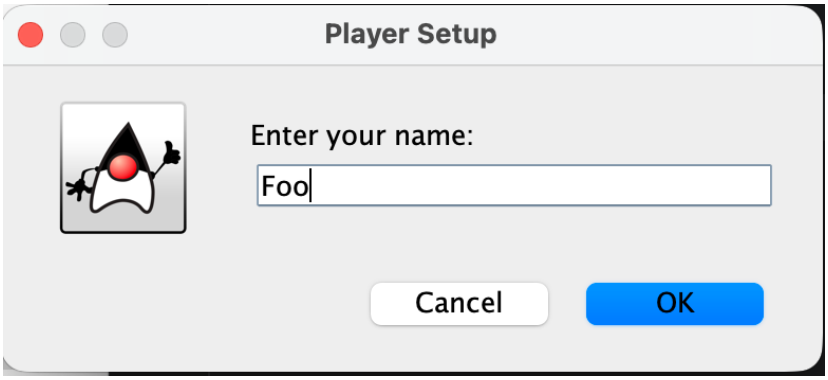
Submission

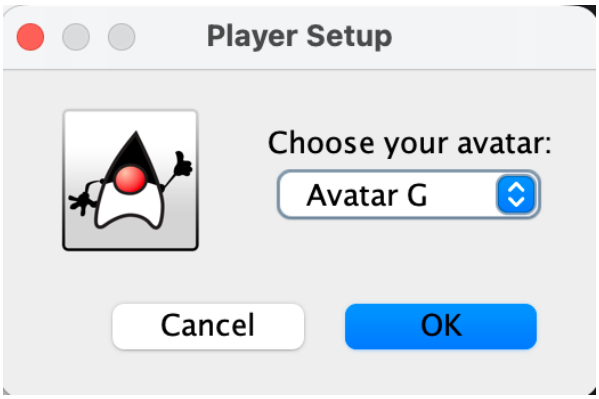
What to Submit: Upload **8 java files** to Gradescope under **PA6 – Connect N**:

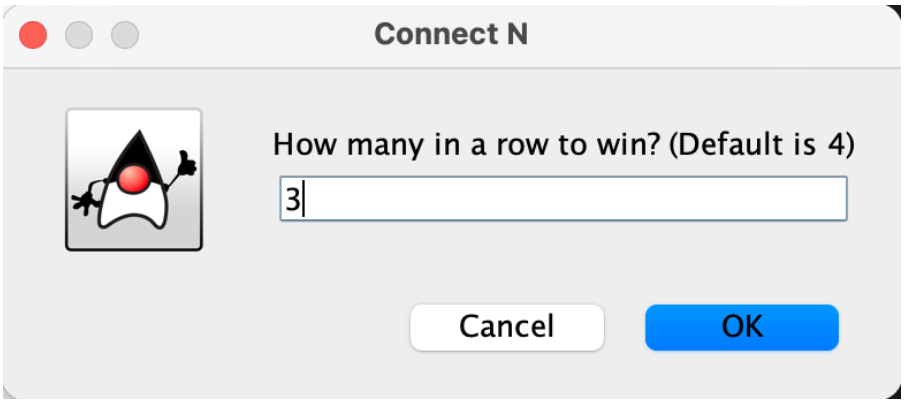
- MyArrayList.java
- MyIterator.java
- Utility.java
- Player.java
- HumanPlayer.java
- ComputerPlayer.java
- Board.java
- Engine.java

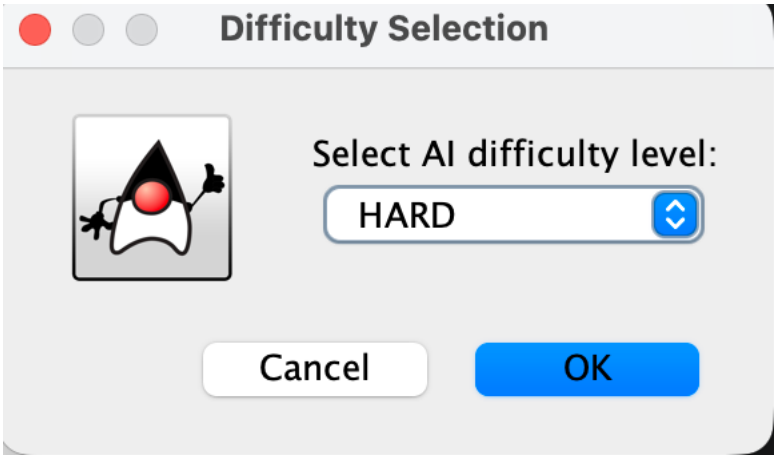
Example of execution

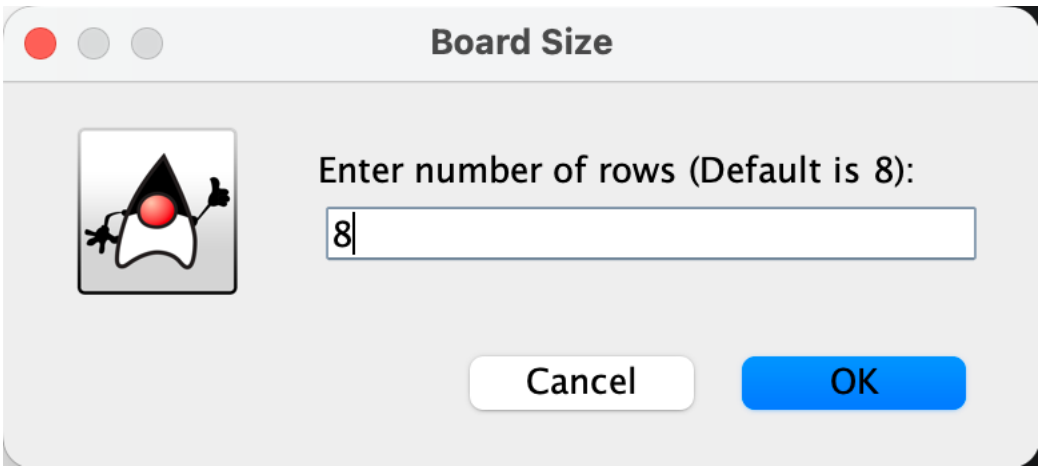
Once you are done implementing all classes. This is what you should see when executing your program.
java ConnectNUI

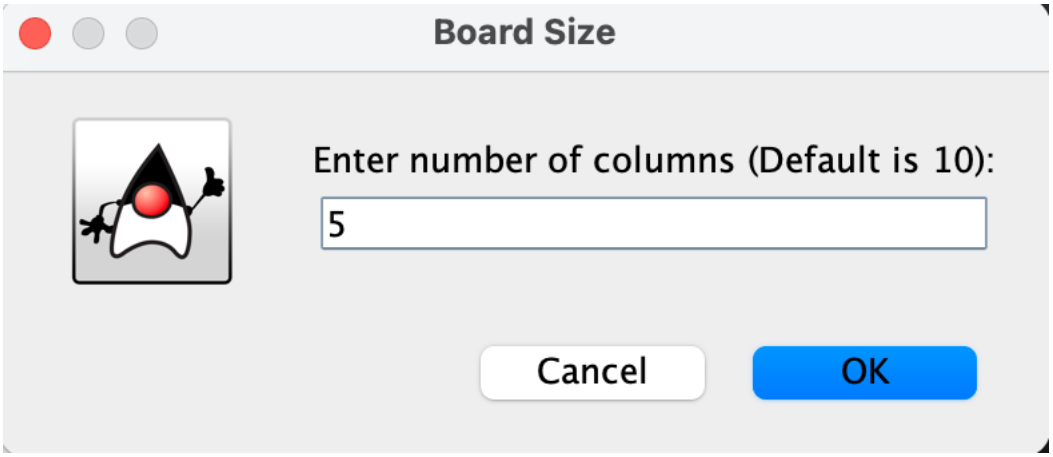


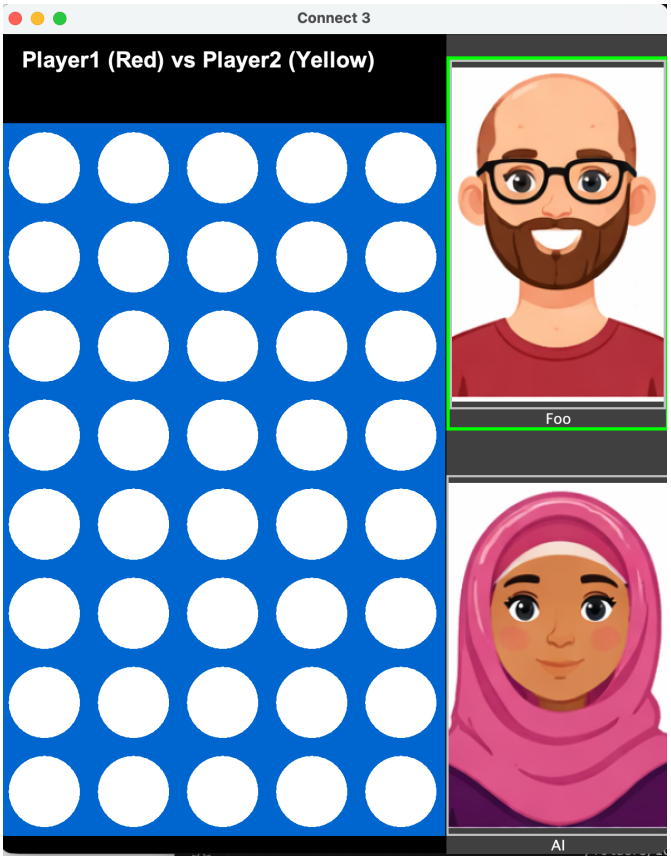




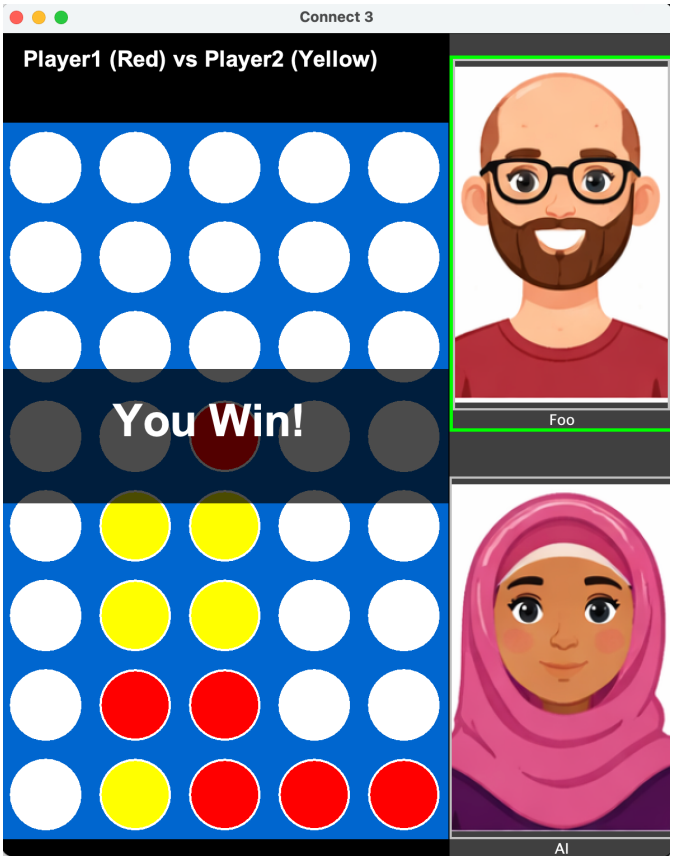

Difficulty Selection
Select AI difficulty level:
HARD
Cancel
OK


Board Size
Enter number of rows (Default is 8):
8
Cancel
OK


Board Size
Enter number of columns (Default is 10):
5
Cancel
OK



The Avatar chosen by the AI Player is random. (You do not have to handle this randomness in your code).



I lost a handful games before I could find a win to proudly share here ! (The settings helped a little bit… with connectN set to 3 only).